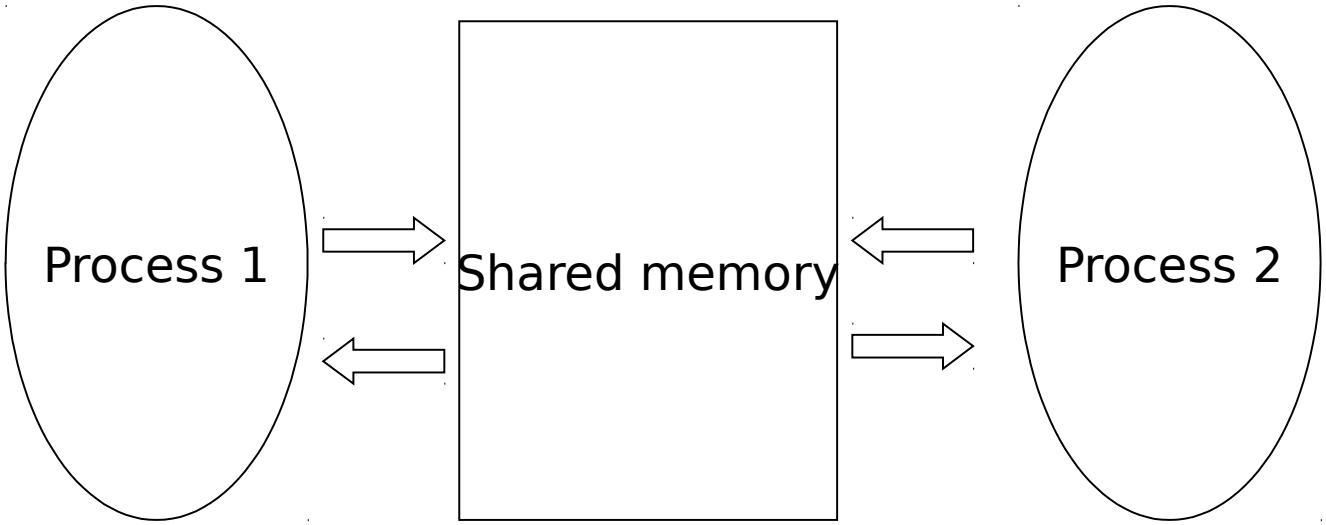
# CSE333 LAB
# Interprocess Communication

Zuhal Altuntaş
[Slides by Dr. Sanem Arslan Yılmaz]

# Shared Memory

- Two or multiple processes share the same memory region.
- The fastest form of IPC




Two processes share the same memory region

# The Memory Model

One process must allocate the segment.

- Then each process desiring to access the segment must attach the segment.
- After finishing its use, each process detaches the segment.
- One process must deallocate the segment.

# Obtain a shared memory id.

```
#include <sys/shm.h>
int shmget(key_t key, size_t size, int flag);
```

**key** – an integer key that specifies which segment to create.
Using **IPC_PRIVATE** as the key value guarantees that a brand new
memory segment is created.

**size** – specifies the number of bytes in the segment. It should be multiple of
the page size.

**flag** – specifies options.

- IPC_CREAT : A new segment should be created.
- IPC_EXCL : This flags always used with IPC_CREAT, causes *shmget* to fail if a segment
- key already exists.
- Mode flags : Permissions of owner, group and others.
- Ex: S_IRUSR, S_IWUSR, S_IROTH, S_IWOTH
-

**Return value:** A segment identifier for success.

# Attachment

```
# include <sys/types.h>
# include <sys/ipc.h>
# include <sys/shm.h>
char *shmat ( int shmid, char *shmaddr, int shmflg )
```

*shmid:* *shared memory identifier*
*shmaddr:* the address at which the shared memory is to be attached to the current
process. This should almost always be a null pointer
*shmflg* :
SHM_RND - indicates that the address specified for the second parameter should be rounded down to a multiple of the page size.
SHM_RDONLY - indicates that the segment will be only read, not written.

*Return Value:* -1 on error,
the address of the attached shared memory segment for success

# Detachment

```
# include <sys/types.h>
# include <sys/ipc.h>
# include <sys/shm.h>

int shmdt ( char *shmaddr)
```

return 0 on success, -1 on error.

# Controlling and Deallocating Shared Memory

```
#include <sys/ipc.h>
#include <sys/shm.h>
int shmctl(int shm_id, int command, struct shmid_ds *buf);
```

*shm_id* : shared memory identifier

*command*:

IPC_STAT - copy the information about the shared memory segment into the buffer *buf*

IPC_RMID - mark the segment as destroyed.

*buf :* save the information of shm_id.

Return 0 on success, -1 on error

# Example

1. shm2.c

# Process Semaphores

Process synchronization is handled by users in shared memory communication.

- Semaphores can be used for synchronizing processes (called process semaphores).
- Process semaphores are allocated, used and deallocated like shared memory segments.
- Although a single semaphore is sufficient for almost all uses, process semaphores come in sets.

# Allocation and Deallocation

- Semaphore allocation: **semget**

```
int semget(key_t key, int nsems, int semflg);
```

  - key: an input key
  - nsems: Number of semaphores to create
  - semflg:  operation | permissions
- Semaphore deallocation and other semaphore controls: **semctl**

```
int semctl(int semid, int semnum, int cmd, ...);
```

  - semid: Semaphore array id
  - semnum:  Index of semaphore in the array
  - cmd:
    - IPC_STAT – Get the semaphore information array into optional arg (sem_num is ignored)
    - IPC_SET – Set the various semaphore array info specified by optional arg  (sem_num is ignored)
    - IPC_RMID – Remove the semaphore array (sem_num is ignored)
    - SETVAL – Set semnum'th semaphore value as specified by optional arg

# Allocating and Deallocating a Binary Semaphore

```
#include <sys/ipc.h>
#include <sys/sem.h>
#include <sys/types.h>

/* We must define union semun ourselves. */
union semun {
        int val;                    // Value for SETVAL
        struct semid_ds *buf;       // Buffer for IPC_STAT & IPC_SET
        unsigned short int *array;  // Array for GETALL, SETALL ...
       };

       /* Obtain a binary semaphore's ID, allocating if necessary. */
       int binary_semaphore_allocation (key_t key, int sem_flags)
       {
              return semget (key, 1, sem_flags);
       }

       /* Deallocate a binary semaphore. All users must have finished their  use. Returns -1 on failure. */
       int binary_semaphore_deallocate (int semid)
       {
             union semun ignored_argument;
             return semctl (semid, 1, IPC_RMID, ignored_argument);
       }
```

# Initializing Semaphores

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/sem.h>

/* We must define union semun ourselves. */
union semun {
        int val;                    // Value for SETVAL
        struct semid_ds *buf;       // Buffer for IPC_STAT & IPC_SET
        unsigned short int *array;  // Array for GETALL, SETALL …
       };

       /* Initialize a binary semaphore with a value of 1. */
       int binary_semaphore_initialize (int semid)
       {
        union semun argument;
        unsigned short values[1];
        values[0] = 1;
        argument.array = values;
        return semctl (semid, 0, SETALL, argument);
       }
```

# Semaphore Operations

- Semaphore usage operations: **semop**

```
int semop(int semid, struct sembuf *sops, unsigned nsops);
```

  - semid: Semaphore array id
  - sops: Semaphore operations to be performed
  - nsops: Number of elements in sops

```
struct sembuf
{
    unsigned short sem_num;     // Index of semaphore to be operated
    short    sem_op;     // Value to be added to the current semaphore value
    short   sem_flg;                    // Operation flags

}
```

- Operation Specifics
  - sem_op > 0 → Value added to the semaphore value
  - sem_op < 0 -> The absolute value of that number is subtracted from semaphore value.
  - sem_op == 0 -> The operation blocks until the semaphore value becomes zero.

- Operation Flags
  - IPC_NOWAIT – Prevents the operation from blocking
  - SEM_UNDO – Kernel automatically undoes the semaphore operation on process exit

# Wait and Post Operations for a Binary Semaphore

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/sem.h>


/* Wait on a binary semaphore. Block until the semaphore value is positive, then  decrement it by 1. */
int binary_semaphore_wait (int semid)
{
        struct sembuf operations[1];
        /* Use the first (and only) semaphore. */
        operations[0].sem_num = 0;
        /* Decrement by 1. */
        operations[0].sem_op = -1;
        /* Permit undo'ing. */
        operations[0].sem_flg = SEM_UNDO;
        return semop (semid, operations, 1);
       }


       /* Post to a binary semaphore: increment its value by 1.  This returns immediately. */
       int binary_semaphore_post (int semid)
       {
        struct sembuf operations[1];
        /* Use the first (and only) semaphore. */
        operations[0].sem_num = 0;
        /* Increment by 1. */
        operations[0].sem_op = 1;
        /* Permit undo'ing. */
        operations[0].sem_flg = SEM_UNDO;
        return semop (semid, operations, 1);
       }
```

# Examples

1. shm_write.c
2. shm_read.c